Reviewer Pack — Minimal Rank of Lie Algebras Bounds DPLL Tree Depth for k-CL…

Entry 2b2480e9f96e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 00:32:05 UTC

Minimal Rank of Lie Algebras Bounds DPLL Tree Depth for k-CLIQUE

Entry ID: `2b2480e9f96e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 00:32:05 UTC

1. Conjecture

Field A (mathematical branch): Lie Algebra Theory **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

> {'sentence_1': 'For every instance of the k-CLIQUE problem, there exists a finite-dimensional Lie algebra L associated with the DPLL search tree for that instance.', 'sentence_2': 'The minimal rank of the Lie algebra L is upper-bounded by a function f(k), where k is the size of the clique.', 'sentence_3': 'Specifically, dim(L) ≤ f(k).'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Lie algebras can encode the structure of complex systems and their transformations, potentially capturing the dynamics of the DPLL search tree.', 'sentence_2': 'The minimal rank provides a measure of simplicity or complexity in the algebraic structure, which might correlate with the depth of the DPLL search tree.', 'sentence_3': 'This conjecture explores an unusual application of Lie algebras to complexity theory, which could lead to new insights into computational problems.'}

Taxonomy category: `LieAlgebraDPLLTree` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `091ecd8fc23cd6f0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all k-CLIQUE instances, the minimal rank of the associated Lie algebra L is less than or equal to f(k) with a support fraction >= 0.9 and the mean difference between dim(L) and f(k) across all seeds ≤ 1.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank Lie algebra" AND "DPLL tree complexity" - "k-CLIQUE problem" AND "Lie algebra dimension" - "finite-dimensional Lie algebra" AND DPLL search tree depth

Top relevant hits considered: - [s2:10.5922/0321-4796-2024-55-2-5] On the dimension of Lie algebrasof infinitesimal affine transformations of direct products of more than two spaces of af - [s2:2309.00098] Conformal Hypergraphs: Duality and Implications for the Upper Clique Transversal Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique_instance(k):
        if k < 3:
            return []
        vertices = list(range(1, k + 1))
        edges = [(i, j) for i in range(1, k + 1) for j in range(i + 1, k + 1)]
        clique_size = random.randint(3, k)
        clique_vertices = random.sample(vertices, clique_size)
        clique_edges = [(i, j) for i in clique_vertices for j in clique_vertices if i < j]
        non_clique_edges = [e for e in edges if e not in clique_edges]
        instance = {'vertices': vertices, 'edges': clique_edges + non_clique_edges}
        return instance

    def dpll_search_tree(instance):
        vertices = instance['vertices']
        edges = instance['edges']

        def dfs(path, remaining_edges):
            if not remaining_edges:
```

```
            return path
        for edge in remaining_edges:
            u, v = edge
            if (u in path and v not in path) or (v in path and u not in path):
                new_path = path + [u] if u not in path else path + [v]
                new_remaining_edges = [e for e in remaining_edges if e != edge]
                result = dfs(new_path, new_remaining_edges)
                if result:
                    return result
        return None

    return dfs([], edges)

def lie_algebra_rank(tree):
    n = len(tree)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for u, v in tree:
        adjacency_matrix[u - 1][v - 1] = 1
        adjacency_matrix[v - 1][u - 1] = 1

    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(n):
            max_row = None
            for j in range(rank, m):
                if matrix[j][i]:
                    max_row = j
                    break
            if max_row is None:
                continue
            matrix[max_row], matrix[rank] = matrix[rank], matrix[max_row]
            rank += 1
            for j in range(m):
                if j != rank - 1:
                    factor = matrix[j][i] / matrix[rank - 1][i]
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[rank - 1][k]
        return rank

    return gaussian_elimination(adjacency_matrix)

def f(k):
    # Placeholder function for the upper bound f(k)
    return k**2

k = random.randint(3, 40)
instance = generate_k_clique_instance(k)
tree = dpll_search_tree(instance)
rank = lie_algebra_rank(tree)
metric_value = rank
conjecture_holds = rank <= f(k)
counterexample = "" if conjecture_holds else f"rank={rank}, f(k)={f(k)}"

return {
    "metric_name": "Lie Algebra Rank",
```

```
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.9 and abs(mean_value - sum(r['metric_value'] for r in results) / le
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7be3198c.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7be3198c.py", line 88, in run_trial
    rank = lie_algebra_rank(tree)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7be3198c.py", line 53, in lie_algebra_r
    n = len(tree)
        ^^^^^^^^^
TypeError: object of type 'NoneType' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that it was unable to provide evidence for or against the conjecture. | next: Re-run the test with appropriate error handling and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11363
2	propose	ollama_remote	glm4:latest	0	0	10539
3	preregistration	ollama_remote	glm4:latest	0	0	5698
4	novelty	ollama_remote	glm4:latest	0	0	4577
5	novelty	ollama_remote	glm4:latest	0	0	5966
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20990
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13505
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13043
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13641
10	judge	ollama_remote	glm4:latest	0	0	8002

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107324 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2b2480e9f96e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2b2480e9f96e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2b2480e9f96e.tar.gz` (if generated)